

Финальный отчёт по проекту Pollify

Веб-сервис проведения опросов и голосований с поддержкой анонимного и неанонимного режимов, множественного выбора, свободных текстовых ответов и кворумной модерации сообществом.

Параметр	Значение
СУБД	PostgreSQL 17
Серверный стек	Go 1.25 (chi, pgx, golang-jwt, bcrypt)
Клиентский стек	React 19 + TypeScript + Vite + react-router-dom
Reverse-proxy	Caddy 2
Контейнеризация	Docker + Docker Compose
Контракт	OpenAPI 3.1 (<code>api/openapi.yaml</code>)
Сущностей в схеме	7
Триггеров	3
Индексов	9
Ограничений CHECK	5
ENUM-типов	2
Транзакций в коде	3
Тестов	21 E2E (Playwright) + интеграционный смюук + unit/repo по компонентам

1. Назначение и бизнес-задача

1.1 Основная бизнес-функция

Pollify информатизирует комплекс взаимосвязанных бизнес-процессов вокруг **сбора мнений в коллективе**:

- Регистрация и аутентификация пользователей** — управление учётными записями с двумя бизнес-ролями.

2. **Создание опросов** — пользователь формулирует вопрос, задаёт варианты ответов и расписание.
3. **Голосование** — другие пользователи отдают свой голос, причём один пользователь может проголосовать в одном опросе только один раз.
4. **Просмотр результатов** — агрегация в виде процентов и счётчиков, опционально — детализация по голосовавшим (только для неанонимных опросов).
5. **Подача жалоб на некорректные опросы** — любой пользователь может пожаловаться, но только один раз на один опрос.
6. **Кворумная модерация** — администраторы рассматривают жалобу; для принятия решения требуется **минимум 2 совпадающих решения** разных администраторов.

1.2 Границы автоматизации

Внутри границ	За пределами (вне scope v1)
создание/чтение опросов	редактирование опросов после старта или появления голосов
голосование	удаление или изменение поданных голосов
агрегация результатов	сложная аналитика (временные ряды)
подача и рассмотрение жалоб	прямое скрытие опроса администратором без жалобы
распределённая модерация (кворум)	административный backdoor
два режима опроса (анонимный / неанонимный)	смешанный режим (options + custom_text одновременно)

1.3 Ключевые сущности предметной области

Пользователь, Опрос, Вариант ответа, Участие в опросе, Голос, Жалоба, Решение администратора.

2. Архитектура клиент-серверного приложения

Приложение реализует трёхзвенную архитектуру с **активным сервером БД**:

```

[ Браузер ] — HTTPS/HTTP — [ Caddy (proxy) ] — HTTP — [ Go API ] — pgx/v5 — [ PostgreSQL 17 ]
  React SPA                      раздаёт /, /api/*                бизнес-логика                триггеры + CHECK
  Bearer JWT                      и валидация                      и валидация                ENUM, FK, PK

```

Прикладной компонент **распределён** между клиентом, application-сервером и СУБД:

Уровень	Что выполняется	Примеры
Клиент (React)	UX-валидация, навигация, локализация	минимальная длина пароля, заполненность обязательных полей, UI-фильтры
Application-сервер (Go)	бизнес-инварианты, авторизация, оркестрация транзакций	проверка кворума, выпуск JWT, маршалинг DTO ↔ домен
СУБД (PostgreSQL)	целостность, ограничения, межтабличные проверки	CHECK, UNIQUE, FK, триггеры, ENUM, транзакции

Такое распределение обеспечивает **defense in depth**: пользователь видит понятную ошибку в UI, но даже при прямом доступе к БД нарушить бизнес-инварианты нельзя — последняя линия обороны декларативная и сидит в каталоге СУБД.

3. ER-модель и схема БД

3.1 Список сущностей и связей

Сущность	Назначение	Атрибуты ключевых типов
users	пользователь системы	id (UUID), email (TEXT UNIQUE), password_hash (TEXT), display_name (TEXT), role (TEXT CHECK), created_at (TIMESTAMPTZ)
polls	опрос	id (UUID), title/description/question (TEXT), is_anonymous/is_multiple_choice/allow_custom_answer/is_hidden (BOOLEAN), max_choices (INTEGER), start_at/end_at/created_at (TIMESTAMPTZ)
options	вариант ответа	id (UUID), poll_id (UUID FK), text (TEXT)
poll_participants	факт участия	poll_id+user_id (составной PK, UUID×UUID), voted_at (TIMESTAMPTZ)
votes	голос	id (UUID), poll_id (UUID FK), option_id (UUID NULL FK), user_id (UUID NULL FK), custom_text (TEXT NULL), created_at (TIMESTAMPTZ)

Сущность	Назначение	Атрибуты ключевых типов
reports	жалоба	id (UUID), poll_id (UUID FK), created_by (UUID FK), reason/comment (TEXT), status (report_status ENUM), approval_count/rejection_count (INTEGER), resolution (TEXT), created_at (TIMESTAMP TZ)
report_reviews	решение администратора	report_id+admin_id (составной PK), decision (review_decision ENUM), comment (TEXT), created_at (TIMESTAMP TZ)

3.2 Типы связей

Связь	Кардинальность	Реализация
polls → options	1:M	options.poll_id FK, ON DELETE CASCADE
polls → reports	1:M	reports.poll_id FK, ON DELETE CASCADE
users → polls (created_by)	1:M	polls.created_by FK
users → reports (created_by)	1:M	reports.created_by FK
users ↔ polls (участие)	M:M	ассоциативная таблица poll_participants с составным PK (poll_id, user_id)
users ↔ polls (голоса)	M:M	votes со ссылками на обе сущности, со специальной обработкой анонимного режима (user_id IS NULL)
reports ↔ users (решения)	M:M	ассоциативная таблица report_reviews с составным PK (report_id, admin_id)

Зачем составной PK на ассоциативной таблице. Он одновременно идентифицирует строку **и** обеспечивает уникальность пары — то есть один и тот же пользователь не может участвовать в одном опросе дважды, и один и тот же админ не может вынести два решения по одной жалобе. Эта избыточность — естественная для M:M, и она встроена в саму структуру ключа.

3.3 Категоризация (супертип / подтипы) — текущее состояние и план

В предметной области присутствует логическая категоризация: пользователь — это супертип, а `USER` и `ADMIN` — два подтипа с разным набором бизнес-функций.

Текущая реализация: одна таблица `users` с полем

`role TEXT NOT NULL CHECK (role IN ('USER', 'ADMIN'))`. Это **single-table inheritance** — все подтипы хранятся в одной таблице, лишние атрибуты подтипов выражаются как nullable-колонки. Преимущества: простота, нет лишних JOIN; недостатки: при появлении атрибутов, специфичных для одного подтипа, схема загромождается.

Перспективное направление (не реализовано): при добавлении атрибутов, специфичных для администраторов (например, область модерации, журнал действий), стоит выделить таблицу `admins(user_id PK FK -> users.id, ...)` — это **disjoint subtype**. Текущий объём данных не оправдывает разделение.

3.4 ER-диаграмма

См. `docs/diagrams/` (ER-диаграмма и Use Case построены отдельно в системном анализе).

4. Типы данных и атрибуты

4.1 Выбор типов

Тип	Где использован	Обоснование
<code>UUID</code>	все первичные ключи и FK	непрерывная глобальная уникальность, безопасно для распределения, не раскрывает порядок и количество строк. Генерация — <code>gen_random_uuid()</code> из <code>pgcrypto</code> (<code>DEFAULT</code> на столбце)

Тип	Где использован	Обоснование
TEXT	строковые поля	в PostgreSQL TEXT, VARCHAR(n) и CHAR(n) имеют одинаковое внутреннее хранение (TOASTed varlena). TEXT не накладывает лимит на длину — лимиты лучше выражать через CHECK либо проверять в приложении, где сообщения об ошибке удобнее настроить
TIMESTAMPTZ	все временные метки (created_at, start_at, end_at, voted_at)	хранит абсолютный момент времени в UTC и автоматически конвертирует при чтении в зависимости от timezone сессии. TIMESTAMP без TZ хранит "плавающее" время и легко приводит к рассинхрону между серверами в разных зонах
BOOLEAN	флаги конфигурации опроса (is_anonymous, is_multiple_choice, allow_custom_answer, is_hidden)	идиоматично, имеет три значения (TRUE/FALSE/NULL) но во всех bool-колонках добавлен NOT NULL
INTEGER	счётчики (max_choices, approval_count, rejection_count)	32-битного хватит

Тип	Где использован	Обоснование
<code>ENUM (report_status, review_decision)</code>	бизнес-перечисления	доменная типизация на уровне СУБД: значение хранится компактно (4 байта), невозможно вставить лишнее значение, можно расширять через <code>ALTER TYPE ... ADD VALUE</code>

4.2 NULL, DEFAULT, UNIQUE, CHECK

Конструкция	Где применена	Зачем
<code>NOT NULL</code>	большинство колонок (все ключи, флаги, обязательные поля)	гарантия отсутствия "трёхзначной" логики там, где она не нужна
<code>NULL</code> (значимый)	<code>votes.user_id</code> , <code>votes.option_id</code> , <code>votes.custom_text</code>	nullable намеренно — это часть бизнес-смысла (см. §5 про анонимность и §6 про XOR-payload)
<code>DEFAULT gen_random_uuid()</code>	PK всех таблиц с <code>id</code>	<code>id</code> вычисляется в БД, приложение не отвечает за уникальность
<code>DEFAULT FALSE</code> , <code>DEFAULT ''</code> , <code>DEFAULT 0</code>	флаги, текстовые поля, счётчики	разумные значения, чтобы <code>INSERT</code> мог опускать колонку
<code>UNIQUE</code>	<code>users.email</code>	<code>email</code> — естественный кандидат-ключ, обеспечивает невозможность двух аккаутов с одним адресом. Зачем при наличии PK по <code>id</code> : PK выбран искусственным (UUID) ради стабильности и анонимности; <code>UNIQUE</code> на <code>email</code> фиксирует деловую уникальность
<code>CHECK</code>	5 ограничений (см. §6)	декларативно описывают инварианты предметной области

5. Анонимность через разделение сущностей

Это главное архитектурное решение проекта, прямо связанное с теорией БД.

5.1 Задача

Обеспечить два инварианта одновременно: 1. **«Один пользователь — один голос в опросе»** — даже при анонимном голосовании. 2. **«Для анонимного опроса нельзя восстановить, кто за что проголосовал»** — даже при наличии полного доступа к схеме.

5.2 Решение

<u>poll_participants</u>	<u>votes</u>
poll_id (FK)	id (PK)
user_id (FK)	poll_id (FK)
voted_at	option_id (NULL для текстового ответа)
PRIMARY KEY (poll_id, user_id)	user_id (NULL для анонимного опроса)
	custom_text (NULL для опросов с вариантами)
	created_at

- Уникальность участия обеспечивается **составным РК**
`poll_participants(poll_id, user_id)` — это структурная гарантия, повторный INSERT приводит к `unique_violation` (SQLSTATE 23505), которое приложение конвертирует в `409 Conflict`.
- Для **анонимных опросов** в `votes.user_id` всегда записывается `NULL`. Связи между голосом и автором попросту нет ни в одной строке.
- Для **неанонимных опросов** в `votes.user_id` записывается id автора, и эндпоинт `GET /api/v1/polls/{id}/results?include_voters=true` может агрегировать голосовавших.

Получается, что **уникальность** (то есть identity-инвариант) и **авторство** (data-инвариант) живут в **двух разных отношениях**. Это позволяет: - сохранить ACID-проверку «один пользователь — один голос» (РК на участии); - разорвать связь `user→vote` на уровне схемы для анонимных опросов (нет колонки → нет данных → нет раскрытия даже под админом).

5.3 Параллель с теорией

Это пример **функциональной декомпозиции отношения**: вместо одного отношения «голос + участник» (где `user_id` нужен и как ключ, и как атрибут) мы выделили его в **проекцию-uniqueness** (`poll_participants`) и **проекцию-data** (`votes`). Проекция-uniqueness защищает один инвариант, проекция-data — другой. Если бы у нас была одна таблица, для анонимности пришлось бы либо нарушать

uniqueness, либо удалять данные после голосования, либо хранить хэш — все варианты хуже.

6. Ограничения целостности

6.1 CHECK constraints (5 штук, все из polls и users и votes)

```
-- polls
CHECK (end_at > start_at)                -- расписание корректно
CHECK ((NOT is_multiple_choice AND max_choices = 0) -- max_choices согласован
      OR (is_multiple_choice AND max_choices > 1))
CHECK (NOT (is_multiple_choice AND allow_custom_answer)) -- режимы взаимоисключающие

-- votes
CHECK ((option_id IS NOT NULL AND custom_text IS NULL) -- XOR payload
      OR (option_id IS NULL AND custom_text IS NOT NULL))

-- users
CHECK (role IN ('USER', 'ADMIN'))          -- enum-like
```

Чем CHECK отличается от UNIQUE. UNIQUE гарантирует отсутствие дубликатов по ключу (одно ограничение — несколько строк), CHECK — корректность отдельной строки или её колонок. CHECK не может ссылаться на другие строки или таблицы (для этого нужен триггер, см. §8).

6.2 FOREIGN KEY и стратегии удаления

FK	Стратегия	Почему
options.poll_id → polls.id	ON DELETE CASCADE	вариант ответа не имеет смысла без опроса
votes.poll_id → polls.id	ON DELETE CASCADE	то же для голосов
votes.option_id → options.id	ON DELETE CASCADE	если вариант удаляется (что v1 не делает), привязанные голоса уходят с ним
votes.user_id → users.id	ON DELETE SET NULL	при удалении пользователя голос остаётся (анонимизируется), агрегат сохраняется
poll_participants.poll_id / .user	ON DELETE CASCADE	участие — продолжение опроса/пользователя

FK	Стратегия	Почему
<code>reports.poll_id</code> / <code>.created_by</code>	ON DELETE CASCADE	жалоба без опроса/автора нерелевантна
<code>report_reviews.report_id</code> / <code>.admin</code>	ON DELETE CASCADE	решение бессмысленно без жалобы или админа

6.3 Сводка по нормальным формам

- **1НФ:** все атрибуты атомарны. Списки опций — отдельные строки в `options`, списки голосов — отдельные строки в `votes`. Композитных значений в одной колонке нет.
- **2НФ:** все неключевые атрибуты функционально зависят от **полного** ключа. Особенно важно для ассоциативных таблиц: `poll_participants.voted_at` зависит от пары `(poll_id, user_id)` целиком, частичной зависимости от `poll_id` или от `user_id` нет.
- **3НФ:** транзитивных зависимостей нет. В `polls` нет колонок вроде «название категории», которые зависели бы от какого-нибудь несуществующего `category_id`. `created_by` — это FK, его атрибуты тянутся через JOIN из `users`, а не дублируются в `polls`.
- **БКНФ:** каждый детерминант — суперключ. Здесь схема сравнительно простая, БКНФ совпадает с 3НФ.
- **4НФ:** многозначных зависимостей нет, поскольку для каждой М:М введена своя ассоциативная таблица.

6.4 Сознательная денормализация

В `reports` хранятся **денормализованные счётчики** `approval_count` и `rejection_count`. Они дублируют `COUNT(*) FROM report_reviews WHERE ...`. Зачем: - кворум-логика читает счётчики в одной строке без JOIN на десятки/сотни review; - список модерации показывает прогресс жалобы без агрегирующего подзапроса на каждый ряд.

Цена — два места записи (`report_reviews` INSERT + `reports` UPDATE), но это всегда атомарно в одной транзакции (\$9).

7. Индексы

Девять индексов поверх первичных:

```
polls(created_by)           -- "Мои опросы"
polls(is_hidden, start_at, end_at) -- список с фильтром по статусу
```

```
options(poll_id) -- JOIN options для деталей
votes(poll_id), votes(user_id) -- агрегации и поиск голоса
reports(poll_id), reports(created_by), reports(status) -- модерация и одна-активная-жалоба
```

Композитный `polls(is_hidden, start_at, end_at)` — выбран потому что фильтры списка опросов почти всегда стартуют с `is_hidden = FALSE` и продолжаются временным диапазоном. PostgreSQL использует первый столбец композита как селективный, а оставшиеся — для дальнейшего сужения. Порядок выбран от менее селективного (`is_hidden`) к более селективному (`end_at`).

Решено **не делать частичные индексы** в v1 — на текущем объёме данных профит был бы синтетическим.

8. Активный сервер БД: триггеры и функции

PostgreSQL выступает в роли **активного сервера БД**: помимо хранения он несёт прикладную логику, которая срабатывает автоматически на события вставки/обновления. Это три PL/pgSQL функции, навешанные на три триггера.

8.1 Функции

```
-- 1. INSERT/UPDATE в report_reviews требует роли ADMIN
CREATE FUNCTION ensure_report_review_admin() RETURNS trigger AS $$
BEGIN
    IF NOT EXISTS (SELECT 1 FROM users WHERE id = NEW.admin_id AND role = 'ADMIN') THEN
        RAISE EXCEPTION 'report review requires ADMIN role';
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- 2. Запрет self-review: админ не может рассматривать собственную жалобу
CREATE FUNCTION ensure_report_review_not_self() RETURNS trigger AS $$
BEGIN
    IF EXISTS (SELECT 1 FROM reports
        WHERE id = NEW.report_id AND created_by = NEW.admin_id) THEN
        RAISE EXCEPTION 'self review is forbidden';
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- 3. Привязка голоса к варианту того же опроса
CREATE FUNCTION ensure_vote_option_matches_poll() RETURNS trigger AS $$
BEGIN
    IF NEW.option_id IS NOT NULL AND NOT EXISTS (
        SELECT 1 FROM options
        WHERE id = NEW.option_id AND poll_id = NEW.poll_id
    ) THEN
```

```

    RAISE EXCEPTION 'vote option does not belong to poll';
END IF;
RETURN NEW;
END;
$$ LANGUAGE plpgsql;

```

8.2 Триггеры (BEFORE INSERT OR UPDATE FOR EACH ROW)

```

CREATE TRIGGER report_reviews_admin_only BEFORE INSERT OR UPDATE ON report_reviews
FOR EACH ROW EXECUTE FUNCTION ensure_report_review_admin();

CREATE TRIGGER report_reviews_no_self_review BEFORE INSERT OR UPDATE ON report_reviews
FOR EACH ROW EXECUTE FUNCTION ensure_report_review_not_self();

CREATE TRIGGER votes_option_matches_poll BEFORE INSERT OR UPDATE ON votes
FOR EACH ROW EXECUTE FUNCTION ensure_vote_option_matches_poll();

```

8.3 Когда триггер уместнее CHECK

CHECK ограничен **строкой** и не может ссылаться на другие таблицы. Все три правила выше — **межтабличные** (опрос ↔ варианты, жалоба ↔ автор, пользователь ↔ роль), поэтому реализованы триггерами.

Триггер срабатывает **даже при прямом доступе** (psql, миграция, ручной INSERT). Поэтому он — это **последняя линия обороны**: приложение валидирует первым (быстрая обратная связь и человекочитаемая ошибка), а триггер ловит то, что обходит приложение.

9. Транзакции и ACID

9.1 Где явно открыты транзакции

В коде три места используют `BEGIN/COMMIT` напрямую — каждое пишет в две или больше таблиц, которые обязаны остаться согласованными:

Операция	Таблицы	Где
Создание опроса	<code>polls</code> + <code>options[]</code>	<code>backend/internal/polls/adapters/postgres/repository.go:24-5</code>
Подача голоса	<code>poll_participants</code> + <code>votes[]</code>	<code>backend/internal/voting/adapters/postgres/repository.go:33-</code>
Решение администратора	<code>report_reviews</code> + <code>reports</code> (счётчики, статус)	<code>backend/internal/moderation/adapters/postgres/repository.go</code>

Все остальные операции — однооператорные (INSERT/UPDATE/SELECT) и атомарны сами по себе, поэтому транзакция не нужна.

9.2 ACID-разбор на примере подачи голоса

```
BEGIN;  
  INSERT INTO poll_participants (poll_id, user_id, voted_at) VALUES (...);  
  -- если запись (poll_id, user_id) уже существует → unique_violation 23505 → ROLLBACK  
  INSERT INTO votes (poll_id, option_id, user_id, custom_text, created_at) VALUES (...);  
  -- (для multi-choice – несколько INSERT)  
COMMIT;
```

- **A (atomicity):** либо записаны и факт участия, и все голоса, либо ничего.
- **C (consistency):** все CHECK, FK и триггеры срабатывают до COMMIT.
- **I (isolation):** дефолтный уровень `READ COMMITTED` (см. §10) гарантирует, что параллельные читатели не увидят полу-записанное состояние.
- **D (durability):** COMMIT возвращает управление только когда WAL зафиксирован на диске.

9.3 Мappings unique-violation на HTTP

Двойной голос или повторное решение админа обрабатываются единообразно: PostgreSQL возвращает SQLSTATE `23505`, репозиторий проверяет код, оборачивает в доменную ошибку (`ErrPollAlreadyVoted` / `ErrReportReviewAlreadyExists`), а HTTP-адаптер маппит её в `409 Conflict` с понятным `error.code`.

Это пример **оптимистического подхода**: блокировки не берутся заранее, конфликт разрешается на этапе фиксации.

10. Уровни изоляции, MVCC, параллельный доступ

10.1 Уровень изоляции по умолчанию

PostgreSQL по умолчанию работает на уровне `READ COMMITTED`:

- читатель видит данные, **зафиксированные** к моменту начала каждой команды;
- грязное чтение (dirty read) невозможно;
- неповторяемое чтение (non-repeatable read) и фантомы — теоретически возможны, но в нашем сценарии безопасны:
- результаты опроса перечитываются заново при каждом запросе — нет требования «дважды одно и то же в одной транзакции»;
- кворум считается над свежим SELECT внутри транзакции с UPDATE; UNIQUE-constraint на `report_reviews(report_id, admin_id)` пресекает гонку «двойной review» структурно.

Повышение до `REPEATABLE READ` или `SERIALIZABLE` не делалось — стоимость не оправдана, поскольку конфликты ловятся через ограничения.

10.2 MVCC

PostgreSQL реализует **многоверсионное управление параллельностью** (MVCC):

- каждая строка имеет невидимые служебные поля `xmin` / `xmax` — id транзакций, которые её создали/удалили;
- читатели работают со «снапшотом» данных на момент команды; писатели не блокируют читателей и наоборот;
- вакуум (`autovacuum`) убирает мёртвые версии строк позже.

Для проекта это значит: модератор может рассматривать жалобу, в то время как другой пользователь публикует новый голос в том же опросе — никто никого не блокирует.

10.3 Конкурирующие сценарии

Сценарий	Что произойдёт
два админа одновременно жмут «Approve» по одной жалобе	оба <code>INSERT INTO report_reviews</code> стартуют параллельно; первый коммит проходит, второй упирается в UNIQUE-violation на <code>(report_id, admin_id)</code> либо на счётчик; приложение конвертирует в 409
пользователь дважды быстро жмёт «Голосовать» (двойная отправка)	первый запрос инсертит в <code>poll_participants</code> , второй упирается в составной РК и получает 409
редактирование опроса после старта	в SQL-слой запрос не доходит — <code>polls</code> service отвергает; даже если дошёл бы, никаких структурных гарантий нет, но <code>is_hidden</code> и <code>start_at</code> не входят в <code>PATCH</code> и приложение это контролирует

10.4 Оптимистическая vs пессимистическая стратегия

В проекте — **оптимистическая**. Никакой `SELECT ... FOR UPDATE` или явных `LOCK TABLE` нет. Конфликты диагностируются через нарушения UNIQUE/CHECK и обрабатываются на уровне приложения как 409. Это работает, потому что наши конфликты редкие и в основном связаны с уникальностью, а не с расчётами на «прочитал — посчитал — записал».

11. Запросы и отчёты

Запросы вынесены в репозитории (`*/adapters/postgres/`). Несколько показательных:

11.1 Список опросов с фильтрами и пагинацией

```
SELECT id, title, description, question, is_anonymous, is_multiple_choice, max_choices,
       allow_custom_answer, is_hidden, created_by, start_at, end_at, created_at
FROM polls
WHERE 1=1
      -- блоки добавляются динамически:
      AND created_by = $1
      AND is_anonymous = $2
      AND is_hidden = FALSE AND start_at <= CURRENT_TIMESTAMP AND end_at > CURRENT_TIMESTAMP
ORDER BY created_at DESC
LIMIT $N OFFSET $M;
```

Композитный индекс `polls(is_hidden, start_at, end_at)` обслуживает фильтр «active» из UI.

11.2 Подгрузка вариантов одним батч-запросом (N+1 fix)

```
SELECT id, poll_id, text
FROM options
WHERE poll_id = ANY($1::uuid[])
ORDER BY poll_id, text, id;
```

Возвращает варианты сразу для всех опросов одной выборкой; приложение группирует по `poll_id` в map. Это устраняет проблему N+1.

11.3 Результаты опроса (CTE + LEFT JOIN + оконная функция)

```
WITH vote_totals AS (
  SELECT COUNT(*)::INTEGER AS total_votes FROM votes WHERE poll_id = $1
)
SELECT o.id, o.text, COUNT(v.id)::INTEGER AS votes_count,
       CASE WHEN vt.total_votes = 0 THEN 0::DOUBLE PRECISION
            ELSE COUNT(v.id)::DOUBLE PRECISION / vt.total_votes::DOUBLE PRECISION * 100
       END AS percentage
FROM options o
CROSS JOIN vote_totals vt
LEFT JOIN votes v ON v.option_id = o.id
WHERE o.poll_id = $1
GROUP BY o.id, o.text, vt.total_votes
ORDER BY o.text, o.id;
```

- **CTE** `vote_totals` вычисляется один раз и переиспользуется во всех строках через `CROSS JOIN`.
- **LEFT JOIN** на `votes` — чтобы варианты без голосов тоже попали в выдачу с `votes_count = 0`.

- **CASE** для деления — защита от деления на ноль (когда никто ещё не проголосовал).

11.4 Аналитический запрос: голосовавшие по неанонимному опросу

```
SELECT p.user_id, COALESCE(u.display_name, ''),
       COALESCE(
         array_remove(
           array_agg(v.option_id::TEXT ORDER BY v.option_id)
           FILTER (WHERE v.option_id IS NOT NULL),
           NULL),
         '{}'::TEXT[]) AS selected_options,
       COALESCE(MAX(v.custom_text) FILTER (WHERE v.custom_text IS NOT NULL), '') AS custom_text
FROM poll_participants p
LEFT JOIN users u ON u.id = p.user_id
LEFT JOIN votes v ON v.poll_id = p.poll_id AND v.user_id = p.user_id
WHERE p.poll_id = $1
GROUP BY p.user_id, u.display_name
ORDER BY u.display_name, p.user_id;
```

Здесь используются `array_agg ... FILTER` (агрегаты с предикатом) и `MAX(...) FILTER` для свёртки нескольких голосов одного пользователя в одну строку. `LEFT JOIN` обеспечивает, что пользователи, чьи строки в `users` удалены (ON DELETE SET NULL), всё равно попадут в выдачу с пустым именем.

11.5 Запросы count для пагинации

Каждый ресурс с пагинацией имеет отдельную функцию `Count(filter)`, которая повторяет WHERE-блок без LIMIT/OFFSET. Это вернее, чем `COUNT(*) OVER()` в основном запросе (который тянул бы дублирующее агрегатное по всем строкам в каждой строке выдачи).

12. Бизнес-роли и интерфейсы

12.1 Роли в системе

Роль	Кто получает	Бизнес-функции
<code>ANONYMOUS</code> (гость)	браузер без JWT	регистрация, логин
<code>USER</code>	автоматически при регистрации	создание опросов, голосование, просмотр результатов, подача жалоб
<code>ADMIN</code>	назначается out-of-band через SQL	всё, что доступно <code>USER</code> , плюс просмотр жалоб и принятие решений

Почему ADMIN не выдаётся через API. Это сознательное проектное решение: админ — это привилегированный оператор, которого назначает владелец инсталляции вручную. Никакая последовательность вызовов API не даёт обычному пользователю повысить себя до администратора.

12.2 Интерфейсы под роли

Фронтенд — единое SPA, но его поведение зависит от роли через React-гарды:

- `<RequireAuth>` — обёртка вокруг приватных маршрутов; гостя отправляет на `/login`.
- `<RequireAdmin>` — дополнительный гард на маршруты `/moderation` и `/reports/{id}`; неадмина отправляет на `/polls`.
- Кнопка «Moderation» в `App.tsx` появляется только при `user?.role === 'ADMIN'`.

Это даёт каждому типу пользователя **свой набор экранов**:

Экран	Гость	USER	ADMIN
Login / Register	✓	(редирект на полки)	(редирект на полки)
Polls list (с фильтрами)	—	✓	✓
Create poll	—	✓	✓
Poll detail + Vote + Report	—	✓	✓
Poll results	—	✓	✓
Moderation queue	—	—	✓
Report detail + Submit decision	—	—	✓

12.3 Сценарии бизнес-процессов

Сценарий 1. «Опросная активность» (роль USER): 1.

`POST /api/v1/auth/register` → выдача JWT. 2. `POST /api/v1/polls` → создание опроса (транзакция: `polls` + `options`). 3. Опрос виден другим пользователям в `GET /api/v1/polls`. 4. `POST /api/v1/polls/{id}/votes` → голосование (транзакция: `poll_participants` + `votes`). 5. `GET /api/v1/polls/{id}/results` → просмотр. 6. При недовольстве — `POST /api/v1/polls/{id}/reports`.

Сценарий 2. «Модерация» (роль ADMIN, требуется 2 админа): 1. Жалоба в

статусе `OPEN`. 2. `admin1` через UI выбирает жалобу, нажимает Approve →

`POST /api/v1/reports/{id}/reviews`. - триггер `report_reviews_no_self_review`

проверяет, что `admin1` не автор жалобы; - триггер `report_reviews_admin_only`

проверяет роль; - `UNIQUE (report_id, admin_id)` предотвратит повторное решение.

3. Статус становится `IN_REVIEW`, `approval_count = 1`. 4. `admin2` так же одобряет. `approval_count = 2` достигает кворума. 5. Сервис модерации синхронно вызывает `polls.Hide(pollID)` (`is_hidden = TRUE`). 6. Статус жалобы → `RESOLVED`, `resolution = 'poll_hidden'`. 7. Опрос больше не возвращается в активных списках, поле `status` карточки опроса равно `hidden`.

13. Безопасность

13.1 Аутентификация

- Пароли хэшируются **bcrypt** (`golang.org/x/crypto/bcrypt`, `cost 10`).
- При логине сравнивается `bcrypt.CompareHashAndPassword`.
- Сессия — **JWT с HMAC-SHA256**:
 - `sub` = id пользователя,
 - `role` = USER / ADMIN,
 - `iat`, `nbf`, `exp` (1 час жизни),
 - подпись секретом из `JWT_SECRET` (в проде — `openssl rand -hex 32`).
- Транспорт — Bearer-токен в заголовке `Authorization`. В проде поверх TLS (Caddy + Let's Encrypt).

13.2 Авторизация

Middleware-стек на запрос:

1. `auth.Middleware(verifier)` — парсит и проверяет JWT, отказывает на 401, иначе кладёт `AuthenticatedUser` в `context.Context`.
2. `auth.RequireRole(users.RoleAdmin)` — на admin-only routes (например, `/api/v1/reports/*`).
3. Доменный слой (`moderation.Service.Review`) повторно проверяет роль перед действием — defense in depth.

13.3 Защита на уровне СУБД

- **CHECK** и **FK** не позволяют записать «нечеловеческие» данные.
 - **Триггеры** охраняют межтабличные правила, которые CHECK не выражает.
 - **UNIQUE** превращает гонки в детерминированные конфликты.
-

14. Миграции и эволюция схемы

- Все DDL живут в `backend/migrations/000001_init.up.sql` и `.down.sql`.

- При старте api автоматически вызывается `platformpg.ApplyMigrations(ctx, pool, "migrations")`.
- Скрипт `up.sql` **идемпотентен**: `CREATE EXTENSION IF NOT EXISTS`, `CREATE TABLE IF NOT EXISTS`, `CREATE OR REPLACE FUNCTION`, `DROP TRIGGER IF EXISTS` + `CREATE TRIGGER`, защита `CREATE TYPE` через `DO $$ ... IF NOT EXISTS (SELECT 1 FROM pg_type WHERE typename='...') THEN CREATE TYPE ... $$`. Повторный запуск безопасен.
- Добавление сущности → новая миграция `000002_xxx.up.sql` + симметричный rollback `000002_xxx.down.sql`. Версионирование — простое числовое.

15. Тестирование

Покрытие — пять уровней, все зелёные:

Уровень	Инструмент	Что проверяет	Где
Domain unit	<code>go test</code> + чистые функции	валидация правил создания опроса, payload голоса, кворум	<code>internal/*/core/*_test.go</code>
HTTP unit	<code>go test</code> + <code>httptest</code> + <code>stub-сервисы</code>	форма запроса/ответа, маппинг ошибок, role guards	<code>internal/*/adapters/http/handlers_test.go</code>
Repository	<code>go test</code> + <code>postgrestd helper</code>	SQL против реального Postgres, отдельная схема на тест	<code>internal/*/adapters/postgres/repository_test.go</code>

Уровень	Инструмент	Что проверяет	Где
Backend integration	<code>go test</code> против реального chi-роутера и реальной БД	сквозной сценарий register → vote → report → quorum → poll hidden	<code>backend/tests/integration/api_test.go</code>
Frontend E2E	Playwright (Chromium headless)	14 + 7 спец'ов: auth, голосование, результаты, модерация, RU- локализация	<code>frontend/tests/e2e/*.spec.ts</code>

Все слои запускаются одной командой `go test ./...` (с переменной `DATABASE_URL`) и `npm run e2e`.

16. Развёртывание

- **Dev:** `docker-compose.yml` (postgres + api) + `npm run dev` (Vite на :5173 с proxy на api).
- **Prod:** `docker-compose.prod.yml` (postgres + api + web=Caddy). Caddy раздаёт собранный SPA и проксирует `/api/*` и `/healthz` на api.
- **Один Makefile-target** `make up` поднимает prod-стек локально и сразу засекает трёх админов (`admin1..3@gmail.com / admin123`) для демонстрации потока модерации.
- TLS через Let's Encrypt — одна переменная окружения `SITE_ADDRESS=your.domain` плюс открытый порт 443, Caddy сам получает и продлевает сертификат.
- Подробности — `DEPLOY.md`.

17. Структура репозитория

```
pollify/
├── api/openapi.yaml           источник правды для контракта
├── backend/
│   ├── cmd/api/main.go
│   ├── internal/
│   │   ├── users / polls / voting / moderation    – бизнес-компоненты (core + adapters)
│   │   ├── platform/{auth, httpserver, postgres, app, ...} – инфраструктура
│   │   └── pkg/apierror/                          – единый ErrorResponse
│   ├── migrations/000001_init.up.sql              – вся DDL
│   ├── tests/integration/api_test.go              – сквозной смоук
│   └── Dockerfile
├── frontend/
│   ├── src/{api, auth, i18n, pages}               – TypeScript SPA с EN/RU
│   ├── tests/e2e/*.spec.ts                       – Playwright
│   ├── Dockerfile, Caddyfile, vite.config.ts
│   └── package.json
├── docker-compose.yml        dev (api + postgres)
├── docker-compose.prod.yml   prod (caddy + api + postgres)
├── Makefile                  up / down / reset / seed-admins / dev-up / ...
├── DEPLOY.md                 инструкции развёртывания
└── docs/                     этот отчёт, диаграммы, ТЗ
```

18. Краткий итог

Pollify реализует **информатизацию бизнес-задачи распределённого голосования с модерацией сообществом**, опираясь на PostgreSQL как **активный сервер БД**:

- 7 сущностей, связи 1:M и M:M, **анонимность как структурное свойство** через декомпозицию на `poll_participants` и `votes`;
- 5 CHECK, 9 индексов, 3 триггера, 2 enum-типа, составные PK на ассоциативных таблицах — все классы инвариантов закрыты декларативно или процедурно на стороне СУБД;
- **3 транзакции** охватывают именно те операции, где несколько таблиц обязаны прийти в согласованное состояние, остальное — одно-операторно и атомарно;
- **3 уровня доступа** (гость / пользователь / администратор) с собственным интерфейсом и собственным набором эндпоинтов;
- **кворумная модерация** реализована комбинацией доменной логики приложения и структурных гарантий БД (UNIQUE + триггеры).